

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב - מואץ

67130 – מבוא למדעי המחשב - רגיל

67108 – מבוא למדעי המחשב לתלמידי תלפיות

מועד א' - סמסטר א' – תשע"ג – 2012-2013

פרופ' דני לישצ'ינסקי, מר ינאי גונצ'רובסקי

תאריך: 29.1.2013

משך הבחינה: שלוש שעות

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (34 נק'), אך הציון המקסימלי הוא 100.
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית), אין צורך לכתוב javadoc.
- פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
- בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

בהצלחה!

שאלה 1 (34 נקודות)

בשאלה זו נדון בעצים בהם לכל קודקוד (node) יכול להיות מספר כלשהו של ילדים. נשים לב שבאופן כללי, כל קודקוד בעץ מהווה בעצמו שורש של תת-עץ המכיל את כל הקודקודים שמתחתיו. לכן, כאשר אנו מדברים על קודקוד נתון בעץ, ניתן לדבר גם על תת-העץ שקודקוד זה הוא השורש שלו.

נגדיר את **העומק של קודקוד** בעץ בתור מספר הקודקודים במסלול בינו לבין שורש העץ (כולל הקודקוד עצמו והשורש). העומק של שורש העץ, לדוגמא, הוא 1. בהנתן עץ, נגדיר את **גובהו של העץ** בתור עומקו של הקודקוד העמוק ביותר בעץ. נתון הממשק הבא המייצג קודקוד בעץ, כאשר בכל קודקוד נשמר **ערך (תווית)** מטיפוס String, אותו ניתן לקבל ע"י קריאה לשיטה `getData()`:

```
public interface TreeNode {
    /**
     * Returns the children of this node in the tree.
     * If there are no children, an array of size 0 is returned.
     * The returned array and its contents must not be changed.
     */
    public TreeNode[] getChildren();

    /** Returns the label string associated with this node. */
    public String getData();

    /** Returns the height of the tree rooted at this node. */
    public int treeHeight();
}
```

א. (9 נקודות) ממשו את השיטה הבאה:

```
public static String findLargestLabel(TreeNode root)
```

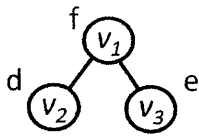
המחזירה את התווית (המוחזרת ע"י `getData()`) אשר ערכה הוא הגדול ביותר מבחינת סדר אלפביתי מבין כל התוויות בעץ הנתון באמצעות `root`. שימו לב כי העץ הנתון הוא לא בהכרח עץ חיפוש (משמע לא בהכרח ממוין). להזכירכם, למחלקה `String` ישנה שיטה בשם `compareTo(String other)` אשר מחזירה ערך חיובי, שלילי או אפס אם המחזרות אשר ממנה מפעילים את השיטה גדולה, קטנה או שווה בהתאמה ל-`other` לפי סדר אלפביתי.

מהי סיבוכיות זמן הריצה של השיטה שמימשתם, בהנחה שכל קריאה לשיטה של העץ הנתון מתבצעת בזמן $O(1)$?

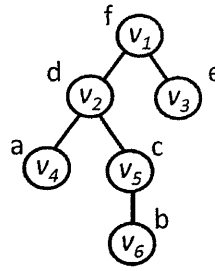
כעת נגדיר מחלקה בשם `DepthLimitedNode` המממשת את הממשק `TreeNode` ולה הבנאי הבא:

```
public DepthLimitedNode(TreeNode sourceRoot, int depthLimit)
```

הבנאי הנ"ל בונה **עץ חדש** אשר כל קודקוד בו מתאים לקודקוד בעץ ששורשו נתון לבנאי באמצעות `sourceRoot`, אך בעץ החדש הזה לא נכללים קודקודים שעומקם עולה על `depthLimit`. פרט לכך, מבנה העץ זהה למבנה העץ המקורי. ראו דוגמא בהמשך. מחלקה זו למעשה מספקת "נקודת מבט" שונה (מוגבלת בעומק) על אותו עץ שניתן כארגומנט לבנאי. הכוונה היא שאובייקט מסוג `DepthLimitedNode` לא יעתיק את המידע (התווית) של כל קודקוד מן העץ המקורי, אלא רק מהווה מעטפת לאובייקט שמממש את הממשק `TreeNode`. כמו כן, אסור לבנאי לשנות את העץ המקורי. ניתן להניח כי מבנה העץ המקורי כפי שהיה בזמן הקריאה לבנאי לא ישתנה בעתיד. כמו כן, ניתן להניח כי הערך הנתון של `depthLimit` הוא לכל הפחות 1. ניתן להניח כי כל קריאה לשיטות של קודקודי העץ המקורי הנתון מתבצעת בזמן $O(1)$.



העץ משמאל הוא עץ מוגבל-עומק כפי שמוגדר לעיל, שנבנה בהנתן קודקוד השורש v_1 בעץ שמימין. בעץ חדש זה, הערך של `depthLimit` הוא 2. קודקודים שעומקם גדול מ-2 בעץ המקורי לא קיימים בעץ החדש.



דוגמא: בעץ שמשמאל לכל קודקוד v_i משויכת אחת מהתוויות (a,b,c,d,e,f) . גובה העץ שבציר השמאלי הוא 4. עומקו של קודקוד שורש העץ v_1 הוא 1, עומקו של הקודקוד v_5 הוא 3.

מכיוון שמדובר רק בנקודות מבט שונה, אם, למשל, התווית של קודקוד v_2 בעץ המקורי תשתנה מ-"d" לתווית חדשה (למשל ל-"new label"), אזי גם בעץ החדש מוגבל-העומק תשתנה בהתאם התווית של הקודקוד המתאים ל-"new label".

- ב. (סה"כ 20 נקודות = 12 עבור הבנאי ו-8 עבור השאר) ממשו את המחלקה `DepthLimitedNode` ע"פ דרישות הסיבוכיות הבאות: אם נסמן ב- n את מספר הקודקודים בעץ הנתון (המקורי), זמן הריצה של הבנאי צריך להיות $O(n)$, ומותר להקצות במהלכו כמות זכרון בסדר גודל של $O(n)$. זמן הריצה של יתר השיטות צריך להיות $O(1)$ ואין להקצות בהן זכרון לאובייקטים חדשים (new). ניתן להניח שאיש לא ינסה לשנות את המערך שמוחזר ע"י `getChildren` וגם לא את תוכנו. הסבירו במילים מדוע המימוש שלכם (של הבנאי ושל שיטות המחלקה) אכן עומד בדרישות סיבוכיות זמן הריצה וסיבוכיות הזכרון שלהלן.

- ג. (5 נקודות) תארו בקצרה מימוש אפשרי למחלקה `DepthLimitedNode` אשר בו מתבצע הבנאי בזמן $O(1)$ אבל זמן הריצה של השיטות האחרות יכול להיות גבוה מ- $O(1)$. ציינו מה יהיה זמן הריצה של כל אחת מן השיטות האחרות. בסעיף זה (בדומה לסעיף ב') ניתן להקצות כמות זכרון עד סדר גודל של $O(n)$.

שאלה 2 (34 נקודות)

נתונה השיטה הרקורסיבית `int mystery(int n)`, כאשר ניתן להניח שהארגומנט שהיא מקבלת הוא תמיד מספר חיובי. סעיפים א' ו-ב' מתייחסים אליה.

```
public static int mystery(int n) {
    return mysteryHelper(0, n);
}
private static int mysteryHelper(int a, int b) {
    if (b < 10) {
        return a + b;
    } else {
        return mysteryHelper(10*(a + b%10), b/10);
    }
}
```

א. (7 נקודות) הבינו ואז הסבירו במשפט אחד מה שיטה זו עושה.

ב. (3 נקודות) בפרט, רשמו במפורש מה יהיה הערך שיוחזר על ידי השיטה עבור הקלט 34917362.

ג. (10 נקודות) כתבו שיטה סטטית רקורסיבית:

```
static public boolean isPalindrome(int[] a)
```

השיטה מקבלת מערך של מספרים שלמים ומחזירה `true` אם ורק אם המערך הוא פלינדרום. מערך פלינדרום הוא מערך שאם עוברים עליו מההתחלה לסוף או מהסוף להתחלה מתקבלים בדיוק אותם הערכים. לדוגמא המערכים $\{1, 382, 7, 382, 1\}$ ו- $\{3, 5, 5, 3\}$ הם מערכי פלינדרום אבל $\{3, 5, 55, 3\}$ איננו כזה.

ד. (10 נקודות) בהינתן מערך בגודל $n \geq 1$, שמכיל רק מספרים שלמים חיוביים ומספר מטרה שלם S , כתבו שיטה אשר מחזירה `true` אם ורק אם ניתן להגיע ל- S באמצעות רצף כלשהו של פעולות חיבור וחיסור של כל המספרים במערך (בכל מספר משתמשים בדיוק פעם אחת). לדוגמא, עבור המערך $\{3, 7, 2\}$ ומספר המטרה 12 השיטה תחזיר `true` היות ש: $3+7+2=12$. בדומה, עבור אותו המערך, השיטה תחזיר `true` גם עבור מספרי המטרה -12 (הרי $-3-7-2=-12$), 8 (הרי $3+7-2=8$), -6 (הרי $3-7-2=-6$). לעומת זאת, עבור מספר המטרה 20 השיטה תחזיר `false`.

ה. (4 נקודות) מהו סדר הגודל של מספר הקריאות הרקורסיביות לשיטה שכתבת בסעיף הקודם, כפונקציה של גודל המערך, n , במקרה הגרוע ביותר? הסבירו בצורה קצרה אך מדויקת (כלומר מבלי לנפנף בידיים).

שאלה 3 (34 נקודות)

במחשבי רשות המחשוב, הן חברי סגל (מורים) והן תלמידים שמורים במחשב באמצעות מחלקות הממשות את הממשק הבא:

```
public interface Person {
    public int getId(); // Returns the ID (תעודת זהות) of this Person.
}
```

(בכל הממשקים והמחלקות בשאלה זו, הושמטו מתודות שאינן רלוונטיות לשאלה, כגון `(Person.getName())` בנוסף, ברשות המחשוב קיימת המחלקה הבאה, אשר מאפשרת למיין באופן יעיל אובייקטים המממשים את `Person` (ההגדרה של `PersonComparator`, המשמשת את מחלקה זו, נמצאת בהמשך העמוד).

```
public class PersonSorter {
    /**
     * Sorts an array of Person according to the order specified
     * by the provided comparator. The array is sorted in-place.
     */
    public static void sort(Person[] people,
                           PersonComparator comparator) {...}
}
```

כאשר הממשק `PersonComparator` אשר מגדיר את הסדר עבור המיון הוא:

```
public interface PersonComparator {
    /**
     * Compares the two given Persons, p1 and p2. Returns a positive
     * number if p1 comes after p2 in the order defined by this
     * PersonComparator, a negative number if p1 comes before p1 in this
     * order, zero if it does not matter which of p1 and p2 comes first.
     */
    public int compare(Person p1, Person p2);
}
```

בשאלה זו תתבקשו לממש מספר שיטות הממיינות אובייקטים בסדרים שונים. שימו לב:

- יש לממש את התשובה לכל סעיף במחלקה חדשה ושונה. שמות המחלקות - לשיקול דעתכם.
- עליכם לממש את השיטות הנ"ל אך ורק באמצעות `PersonSorter.sort` ובאמצעות `PersonComparator` מתאים שתממשו בכל סעיף. פתרונות שימיינו באמצעות קוד מיון שתכתבו או באמצעות שיטות סטנדרטיות של ג'אווה לא יקבלו ניקוד.
- אין להשתמש ביכולות של ג'אווה שלא נלמדו בקורס. (בפרט, למי שמכיר, אין להשתמש ב-Anonymous Classes).

א. (8 נק') ממשו את השיטה

```
public static void sortById(Person[] people)
```

אשר מקבלת מערך של אובייקטים מסוג `Person` וממיינת אותו בסדר עולה של מספרי תעודת הזהות.

ב. (12 נק') סטודנטים מיוצגים במערכת המחשוב באמצעות המחלקה הבאה:

```
public class Student implements Person {
    /** Returns the grade of the student in a given course,
     * or null if the Student did not take this course */
    public Integer getCourseGrade(int courseNumber) { ... }
    // Other data and methods, including implementation of all Person methods.
}
```

ממשו את השיטה:

```
public static void sortDescendingByGrade(Student[] students,
                                          int courseNumber)
```

אשר מקבלת מערך של אובייקטים מסוג **Student** ומס' קורס וממיינת אותו בסדר יורד לפי ציוני התלמידים בקורס הנ"ל, כאשר סטודנטים שלא קיבלו ציון בקורס הנ"ל יופיעו בסוף המערך. סטודנטים אשר קיבלו כולם את אותו הציון ימוינו בסדר עולה של מספרי תעודות הזהות שלהם, וכך גם כל הסטודנטים אשר לא קיבלו ציון. (פתרון אשר פועל נכון רק כשלכל הסטודנטים ציון בקורס יקבל ניקוד חלקי).

ג. (8 נק') מורים מיוצגים במערכת המחשוב באמצעות המחלקה הבאה:

```
public class Teacher implements Person {
    public int[] getCourses(){ ... } // You won't need to use this method
    // Other data and methods, including implementation of all Person methods.
}
```

באוניברסיטה ישנם סטודנטים המשמשים גם כמורים. לכל סטודנט כנ"ל מתאימים שני אובייקטים שונים במערכת: אחד מסוג **Student** ואחד מסוג **Teacher**. כתבו את השיטה:

```
public static int countStudentTeachers(Student[] allStudents,
                                         Teacher[] allTeachers)
```

המקבלת מערך עם כל התלמידים שבמערכת ומערך עם כל המורים שבמערכת ומחזירה את מספר התלמידים המשמשים גם כמורים. שימו לב:

1. המערכים הנתונים אינם ממויינים ואסור לשיטה שתכתבו לשנותם. עם זאת, אתם רשאים להקצות מערך אחד נוסף.
2. בהנחה שיש מספר דומה של סטודנטים ומרצים – נסמנו ב- n , ודאו שהפתרון שלכם רץ בסיבוכיות זמן ממש יותר נמוכה מ- $O(n^2)$.
3. לסעיף זה קיים פתרון קצר מאוד (השתמשו בכלים העומדים לרשותכם). פתרון ארוך או סבוך יתר על המידה לא יקבל ניקוד מלא.

ד. (6 נק') איזו מגבלה של שפת ג'אווה מונעת הגדרה של אובייקט אחד אשר הינו גם מטיפוס **Student** וגם מטיפוס **Teacher**, כפי שהטיפוסים הללו הוגדרו להלן? הציעו (במילים - אין צורך לממש) דרך לשנות את איפיון המערכת באופן שיאפשר לשמור אובייקט יחיד עבור כל סטודנט המשמש כמורה. ודאו שהקוד שכתבתם עבור סעיפים א'-ג' עובד ללא כל שינוי בקוד, גם לאחר שינוי אפיון המערכת שהצעתם.

שאלה 4 (34 נקודות)

במזרח הרחוק, כידוע, הכל הפוך. מורה בקורס מבוא למחשבי המדע במזרח הרחוק לימד בכיתה את אלגוריתם SortMerge למיון מערכים. להלן תיאור האלגוריתם - לצורך הפשטות אנו מתארים את האלגוריתם אך ורק עבור מערכים שגודלם הוא חזקה של 2 (כלומר $1/2/4/8/16/\dots$):

1. ממיינים את האיבר הראשון והשני כך שהם בסדר עולה. ממיינים את האיבר השלישי והרביעי כך שהם בסדר עולה וכו'...
2. מבצעים מיזוג (merge) לאיברים הראשון והשני עם האיברים השלישי והרביעי. מבצעים מיזוג לאיברים החמישי והשישי עם האיברים השביעי והשמיני וכו'...
3. מבצעים מיזוג (merge) לאיברים הראשון עד הרביעי עם האיברים החמישי עד השמיני. מבצעים מיזוג לאיברים התשיעי עד השנים עשר עם האיברים שלושה עשר עד השישה עשר וכו'...
4. וכך הלאה, עד הצעד האחרון, בו מבצעים מיזוג לחציו הראשון של המערך עם חציו השני של המערך.

בשאלה זו (סעיפים א' וד') מותר להקצות כמות זכרון בסדר גודל של $O(n)$, כאשר n הוא גודל המערך למיון.

א. (14 נקודות) כתבו מחלקה בשם `SortMerge` ובתוכה שיטה סטטית בשם `sortMerge` המקבלת מערך (בגודל שהינו חזקה של 2) של `int` וממיינת אותו באמצעות האלגוריתם שלעיל (שימו לב: עליכם לכתוב גם את הקוד שמבצע את המיזוג). על השיטה להמנע מקריאות רקורסיביות.

ב. (6 נקודות) מהי סיבוכיות הזמן של השיטה שכתבתם בסעיף א', כתלות בגודל המערך שניתן לה כקלט? נמקו.

ג. (6 נקודות) גם אלגוריתם המיון MergeSort שנלמד בכיתה מבצע מיזוגים על מנת למיין איברים של מערך. עבור מערך בן 8 איברים, תארו את המיזוגים אותם מבצע MergeSort, לפי סדר הביצוע שלהם. הסבירו מדוע ניתן לטעון כי האלגוריתמים שקולים (מספיק להתייחס רק למקרה של מערך בן 8 איברים).

ד. (8 נקודות) בתוך המחלקה `SortMerge`, כתבו שיטה סטטית נוספת בשם `sort` המקבלת מערך של `int` שהינו בגודל כלשהו (לאו דווקא חזקה של 2) וממיינת אותו באמצעות שימוש בשיטה שכתבתם בסעיף א' (עליכם להשתמש בשיטה מסעיף א' כמו שהיא ולא לשנותה). על פתרוןכם לשמור על סיבוכיות הזמן שמצאתם בסעיף ב' – נמקו מדוע זה אכן המצב.